

TEMA 7. Aspectos funcionales

1. ¿Qué es un puntero a una función? Pon un ejemplo de código en C, donde se define una función y que reciba una cadena de caracteres como parámetro y devuelva la cadena en mayúsculas. Crea un puntero en una variable local a dicha función llamado `aMayusculas` e invócala con el puntero.

✓ ¿Qué es un puntero a una función?

Un **puntero a una función** es una variable que **almacena la dirección de memoria de una función**.

Esto permite **invocar la función indirectamente**, pasarla como parámetro, devolverla desde otra función o decidir **en tiempo de ejecución** qué función ejecutar.

👉 En C, las funciones **no son objetos**, pero **sí se pueden referenciar mediante punteros**.

✓ Ejemplo en C

Vamos a:

1. Definir una función que recibe una cadena (`char *`)
 2. Devuelve la misma cadena convertida a **mayúsculas**
 3. Crear un **puntero a función** llamado `aMayusculas`
 4. Invocar la función **a través del puntero**
-

◆ Función que convierte una cadena a mayúsculas

```
#include <stdio.h>
#include <ctype.h>

char* convertirAMayusculas(char* texto) {
    for (int i = 0; texto[i] != '\0'; i++) {
        texto[i] = toupper((unsigned char)texto[i]);
    }
    return texto;
}
```

◆ Uso de un puntero a función

```
int main() {

    // Declaración del puntero a función
    char* (*aMayusculas)(char*);

    // Asignamos la función al puntero
```

```
aMayusculas = convertirAMayusculas;

char cadena[] = "Hola mundo";

// Invocación a través del puntero
char* resultado = aMayusculas(cadena);

printf("%s\n", resultado);

return 0;
}
```

✓ Salida del programa

HOLA MUNDO

💡 Explicación clave del puntero

```
char* (*aMayusculas)(char*);
```

Se lee como:

"aMayusculas es un puntero a una función que recibe un char* y devuelve un char*".

Y la llamada:

```
aMayusculas(cadena);
```

es equivalente a:

```
convertirAMayusculas(cadena);
```

✓ Idea clave para examen

- Un **puntero a función** almacena la dirección de una función.
- Permite **invocación indirecta**.
- Es muy usado en:
 - *callbacks*
 - tablas de funciones
 - programación modular

- librerías estándar (por ejemplo, `qsort`)

👉 Frase redonda:

Un puntero a función permite tratar a las funciones como valores y decidir dinámicamente cuál ejecutar.

2. ¿Qué es una **función lambda** en un lenguaje de programación? Pon un ejemplo similar al anterior en Javascript y otro en Java con funciones lambda. Usa una variable local `aMayusculas` para apuntar a la función lambda. Por simplicidad, en Java, emplea `Function<String, String>` para el tipo de la referencia a la función lambda.

✅ ¿Qué es una **función lambda**?

Una **función lambda** es una **función anónima** (no tiene nombre propio) que se puede:

- asignar a una variable,
- pasar como argumento,
- devolver como resultado.

Su objetivo principal es **tratar las funciones como valores**, de forma compacta y expresiva, sin necesidad de definir una función tradicional completa.

✅ Ejemplo en **JavaScript**

En JavaScript, las funciones lambda se escriben con la **arrow function** (`=>`).

Función lambda que pasa una cadena a mayúsculas

```
let aMayusculas = (texto) => texto.toUpperCase();

// Uso
let resultado = aMayusculas("Hola mundo");
console.log(resultado);
```

Salida

```
HOLA MUNDO
```

♦ Aquí:

- `aMayusculas` es una **variable local**
- La función es **anónima**
- Se asigna directamente a la variable
- Puede invocarse igual que una función normal

✓ Ejemplo en **Java** con funciones lambda

En Java, las lambdas se introducen a partir de **Java 8** y se usan junto con **interfaces funcionales**. Por simplicidad, usamos `Function<String, String>`.

Definición de la lambda

```
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {

        Function<String, String> aMayusculas =
            texto -> texto.toUpperCase();

        String resultado = aMayusculas.apply("Hola mundo");
        System.out.println(resultado);
    }
}
```

Salida

```
HOLA MUNDO
```

🔍 Detalles importantes en Java

```
Function<String, String>
```

Significa:

- recibe un `String`
- devuelve un `String`

La lambda:

```
texto -> texto.toUpperCase()
```

es la implementación del método abstracto:

```
String apply(String texto);
```

✦ La llamada se hace con:

```
aMayusculas.apply("Hola mundo");
```

✓ Comparación rápida con punteros a función (C)

Lenguaje	Cómo se representa
C	Punteros a función
JavaScript	Funciones como objetos
Java	Lambdas + interfaces funcionales

👉 Las lambdas son la **evolución moderna y segura** de los punteros a función.

🗨️ Resumen perfecto para examen

- Una **función lambda** es una función anónima tratada como valor.
- Se puede asignar a variables y pasar como parámetro.
- En **JavaScript**, las lambdas usan `=>`.
- En **Java**, las lambdas implementan interfaces funcionales.
- `Function<String, String>` representa una función de `String → String`.
- Las lambdas facilitan programación funcional y código más expresivo.

👉 **Frase redonda para cerrar:**

Las funciones lambda permiten expresar comportamiento como datos, de forma concisa y segura.

3. ¿Qué es el **paradigma funcional**? ¿Por qué a algunos lenguajes orientados a objetos como Java 8, se les llama multi-paradigma? ¿Qué quiere decir que las funciones son "ciudadanos de primera clase"?

✓ ¿Qué es el **paradigma funcional**?

El **paradigma funcional** es un estilo de programación en el que el programa se construye **evaluando funciones**, de forma similar a las **funciones matemáticas**.

Sus ideas fundamentales son:

- ✓ **Funciones puras**: el resultado depende solo de sus parámetros
- ✓ **Ausencia de efectos secundarios** (no se modifica el estado externo)
- ✓ **Inmutabilidad** de los datos
- ✓ Uso intensivo de **funciones de orden superior** (funciones que reciben o devuelven otras funciones)
- ✓ Enfoque **declarativo**: se describe *qué* se quiere hacer, no *cómo*

Ejemplo conceptual:

```
datos → transformar → filtrar → combinar → resultado
```

Lenguajes típicamente funcionales:

- Haskell
- Lisp / Scheme
- Erlang
- OCaml

✓ ¿Por qué lenguajes como **Java 8** se consideran **multi-paradigma**?

Un lenguaje **multi-paradigma** es aquel que **admite varios estilos de programación**, no uno solo.

Java nació como un lenguaje:

- **Orientado a objetos**
- **Imperativo**
- Basado en **estado mutable**

Desde **Java 8**, incorpora rasgos **funcionales**, como:

- ✓ Funciones lambda
- ✓ Interfaces funcionales (**Function**, **Predicate**, etc.)
- ✓ Streams (**map**, **filter**, **reduce**)
- ✓ Programación declarativa sobre colecciones

Ejemplo en Java 8:

```
lista.stream()
    .filter(x -> x > 0)
    .map(x -> x * 2)
    .forEach(System.out::println);
```

👉 Java **no deja de ser orientado a objetos**, pero **adopta el paradigma funcional** cuando conviene.

Por eso se dice que Java es:

un lenguaje multi-paradigma: imperativo + orientado a objetos + funcional.

✓ ¿Qué significa que las funciones sean **“ciudadanos de primera clase”**?

Decir que las funciones son **ciudadanos de primera clase** significa que las funciones pueden tratarse **igual que cualquier otro valor del lenguaje**, como un número o un objeto.

Concretamente, una función puede:

- ✓ Asignarse a una variable

-  Pasarse como argumento a otra función
-  Devolverse como resultado
-  Almacenarse en estructuras de datos

Ejemplo conceptual (Java 8):

```
Function<String, String> f = s -> s.toUpperCase();
```

Aquí:

- La función se asigna a una variable
- Se pasa como valor
- Se invoca dinámicamente

Esto **no era posible en Java antes de Java 8** sin interfaces anónimas mucho más verbosas.

-  Relación entre paradigma funcional y ciudadanos de primera clase

 **El paradigma funcional se basa en que las funciones sean ciudadanos de primera clase.**

Sin eso:

- No existirían lambdas
- No existirían `map`, `filter`, `reduce`
- No podría componerse comportamiento

-  Comparación rápida

Lenguaje	¿Funciones como ciudadanos de primera clase?
C	 Parcial (punteros a función)
Java ≤ 7	 No
Java ≥ 8	 Sí (lambdas)
JavaScript	 Sí
Haskell	 Sí (totalmente funcional)

-  Resumen perfecto para examen

- El **paradigma funcional** basa el programa en la composición y evaluación de funciones, evitando efectos secundarios.
- Java 8 se considera **multi-paradigma** porque combina POO con características funcionales.
- Que las funciones sean **ciudadanos de primera clase** significa que pueden tratarse como valores normales: asignarse, pasarse y devolverse.

- Las lambdas y los streams introducen programación funcional **sin abandonar la orientación a objetos**.

👉 **Frase redonda para cerrar:**

Un lenguaje es funcional cuando permite tratar funciones como valores, y es multi-paradigma cuando combina este enfoque con otros estilos sin imponer uno único.

4. Explica la sintaxis básica de una función lambda en Java.

En **Java**, una **función lambda** es una forma **compacta** de expresar la implementación de una **interfaz funcional** (una interfaz con **un único método abstracto**).

✅ Sintaxis básica de una lambda en Java

La forma general es:

```
(parámetros) -> expresión
```

o, si hay más de una instrucción:

```
(parámetros) -> {  
    instrucciones;  
    return valor;  
}
```

◆ 1. Parámetros

Un solo parámetro (sin paréntesis opcionalmente)

```
x -> x * 2
```

Equivale a:

```
(int x) -> x * 2
```

(Java suele **inferir el tipo**).

Varios parámetros (paréntesis obligatorios)

```
(a, b) -> a + b
```

Sin parámetros

```
() -> System.out.println("Hola")
```

◆ 2. Flecha ->

La flecha separa:

- a la **izquierda** → los parámetros
- a la **derecha** → el cuerpo de la función

Es el operador que **define la lambda**.

◆ 3. Cuerpo de la lambda

Expresión única (retorno implícito)

```
s -> s.toUpperCase()
```

No hace falta escribir **return**.

Bloque de instrucciones (retorno explícito)

```
s -> {  
    String res = s.toUpperCase();  
    return res;  
}
```

Aquí:

- Las llaves **{ }** son obligatorias
- **return** es obligatorio si se devuelve un valor

✅ Relación con interfaces funcionales

Una lambda **siempre implementa** el método abstracto de una **interfaz funcional**.

Ejemplo con `Function<T, R>`:

```
Function<String, String> aMayusculas =  
    s -> s.toUpperCase();
```

`Function<String, String>` define:

```
String apply(String s);
```

La lambda **implementa ese método**.

✓ Ejemplo completo

```
import java.util.function.Function;  
  
public class Main {  
    public static void main(String[] args) {  
  
        Function<String, String> aMayusculas =  
            texto -> texto.toUpperCase();  
  
        System.out.println(aMayusculas.apply("Hola mundo"));  
    }  
}
```

💡 Equivalencia con clases anónimas (para entenderla mejor)

Esta lambda:

```
s -> s.toUpperCase()
```

equivale conceptualmente a:

```
new Function<String, String>() {  
    @Override  
    public String apply(String s) {  
        return s.toUpperCase();  
    }  
};
```

👉 La lambda es **más corta, más clara y más expresiva**.

✅ Resumen perfecto para examen

- Una lambda en Java tiene la forma:
(**parámetros**) -> **cuerpo**
- El cuerpo puede ser:
 - una **expresión** (return implícito)
 - o un **bloque** (return explícito)
- Java **infiere los tipos** de los parámetros
- Las lambdas **implementan interfaces funcionales**
- Son la base de la programación funcional en Java 8+

👉 **Frase redonda final:**

*Una función lambda en Java es una expresión que implementa un método funcional de forma concisa mediante la sintaxis (**parámetros**) -> **cuerpo**.*

5. Ahora recibamos una función como parámetro a un método y la llamaremos desde dentro. Amplia los ejemplos anteriores de Java y JavaScript con un método llamado **transformar**, que reciba un **String** como parámetro y luego una función transformadora como lo es **aMayúsculas** y la invoque desde dentro.

Vamos a **ampliar los ejemplos anteriores** para mostrar claramente **cómo pasar una función como parámetro** y **ejecutarla desde dentro de otro método**, tanto en **JavaScript** como en **Java**. Este es uno de los pilares del **paradigma funcional: funciones de orden superior**.

✅ Idea general

Queremos un método:

```
transformar(String texto, función transformadora)
```

- Recibe un **String**
 - Recibe una **función** que transforma ese **String**
 - Llama a esa función desde dentro
 - Devuelve el resultado transformado
-

✅ Ejemplo en JavaScript

En JavaScript, las funciones son **ciudadanos de primera clase**, así que el ejemplo es muy directo.

Función `transformar`

```
function transformar(texto, transformadora) {  
  return transformadora(texto);  
}
```

Función lambda `aMayusculas`

```
let aMayusculas = (texto) => texto.toUpperCase();
```

Uso completo

```
let resultado = transformar("Hola mundo", aMayusculas);  
console.log(resultado);
```

Salida

HOLA MUNDO

Qué está pasando

- `transformar` **recibe una función**
- Esa función se almacena en el parámetro `transformadora`
- Se invoca como cualquier otra función:

```
transformadora(texto)
```

✅ JavaScript permite pasar funciones **sin ningún tipo auxiliar**.

Ejemplo en Java

En Java necesitamos una **interfaz funcional** para representar la función.
Usamos `Function<String, String>`.

Método `transformar`

```
import java.util.function.Function;

public class Util {

    public static String transformar(
        String texto,
        Function<String, String> transformadora) {

        return transformadora.apply(texto);
    }
}
```

Función lambda `aMayusculas`

```
Function<String, String> aMayusculas =
    s -> s.toUpperCase();
```

Uso completo

```
public class Main {

    public static void main(String[] args) {

        Function<String, String> aMayusculas =
            s -> s.toUpperCase();

        String resultado =
            Util.transformar("Hola mundo", aMayusculas);

        System.out.println(resultado);
    }
}
```

Salida

```
HOLA MUNDO
```

Qué está pasando en Java

- `Function<String, String>` representa:

```
String f(String s)
```

- La lambda implementa ese método
- **transformar** recibe comportamiento como parámetro
- La llamada se hace con:

```
transformadora.apply(texto);
```



Comparación JavaScript vs Java

Aspecto	JavaScript	Java
¿Funciones como valores?	✓ Nativo	✓ Desde Java 8
Tipo explícito	✗ No	✓ <code>Function<T,R></code>
Llamada	<code>f(x)</code>	<code>f.apply(x)</code>
Paradigma funcional	✓ Total	✓ Parcial (multi-paradigma)



Concepto clave: funciones de orden superior

Ambos ejemplos ilustran una **función de orden superior**:

Una función que **recibe otra función como parámetro** o **devuelve una función**.

Este concepto es **fundamental** en:

- Programación funcional
- Callbacks
- Streams (`map`, `filter`, `reduce`)
- APIs modernas



Resumen perfecto para examen

- Pasar una función como parámetro permite **parametrizar el comportamiento**.
- En **JavaScript**, las funciones se pasan directamente.
- En **Java**, se usan **interfaces funcionales** y lambdas.
- El método `transformar` es una **función de orden superior**.
- Este mecanismo es la base del **paradigma funcional moderno**.



Frase redonda final:

6. Ahora, invoca **transformar**, con una nueva función lambda directamente en la llamada a **transformar**, por ejemplo, una función lambda que invierta la cadena. Define la función de inversión justo cuando la estás pasando como parámetro.

Podemos **definir el comportamiento justo en la llamada**, sin crear variables auxiliares.

✓ Idea clave

En lugar de:

```
definir función → guardarla → pasarla
```

hacemos directamente:

```
pasar una lambda directamente como argumento
```

✓ Ejemplo en JavaScript

Recordemos la función **transformar**:

```
function transformar(texto, transformadora) {  
  return transformadora(texto);  
}
```

✓ Invocación con una **lambda inline** que invierte la cadena

```
let resultado = transformar(  
  "Hola mundo",  
  (texto) => texto.split("").reverse().join("")  
);  
  
console.log(resultado);
```

Salida

```
odnum aloH
```

✓ La función de inversión:

- **no tiene nombre**
- **no se guarda en ninguna variable**
- se define **exactamente donde se usa**

✓ Ejemplo en Java

Recordemos el método `transformar`:

```
import java.util.function.Function;

public class Util {

    public static String transformar(
        String texto,
        Function<String, String> transformadora) {

        return transformadora.apply(texto);
    }
}
```

✓ Invocación con una **lambda inline** que invierte la cadena

```
public class Main {

    public static void main(String[] args) {

        String resultado = Util.transformar(
            "Hola mundo",
            s -> new StringBuilder(s).reverse().toString()
        );

        System.out.println(resultado);
    }
}
```

Salida

```
odnum aloH
```

🔍 Qué está pasando exactamente

En Java:

```
s -> new StringBuilder(s).reverse().toString()
```

- `s` es el parámetro `String`
- La lambda implementa `Function<String, String>`
- Se pasa **directamente como argumento**
- No se necesita ninguna variable intermedia

La llamada es equivalente a:

```
Function<String, String> invertir = s -> new  
StringBuilder(s).reverse().toString();  
Util.transformar("Hola mundo", invertir);
```

pero **mucho más concisa**.

✓ Comparación rápida

Lenguaje	Definir función inline	Guardar función
JavaScript	✓ Muy natural	✓ Opcional
Java	✓ Desde Java 8	✓ Opcional

✓ Concepto clave reforzado

Esto demuestra que:

- ✓ Las funciones son **ciudadanos de primera clase**
- ✓ `transformar` es una **función de orden superior**
- ✓ El comportamiento se decide **en tiempo de llamada**
- ✓ El código es más expresivo y flexible

🧠 Resumen perfecto para examen

- Las funciones lambda pueden definirse **directamente en la llamada** a un método.
- Esto evita crear variables auxiliares innecesarias.
- En JavaScript, se usan arrow functions (`=>`).
- En Java, se usan lambdas compatibles con interfaces funcionales.
- Esta técnica es fundamental en programación funcional y APIs modernas.

👉 **Frase redonda final:**

Definir una lambda directamente en la llamada permite expresar el comportamiento justo donde se necesita, haciendo el código más claro y declarativo.

7. ¿Qué se entiende por cierre o "closure" en el contexto de las funciones lambda? Pon un ejemplo en Java de cómo una función lambda es capaz de acceder a una variable local en el contexto donde fue definida. Modifica el ejemplo anterior, creando otra función lambda para transformar una cadena, pero que lo que haga es concatenar a la cadena de entrada otra cadena que está en una variable local definida fuera de la función lambda.

✅ ¿Qué es un **cierre (closure)** en el contexto de funciones lambda?

Un **cierre (closure)** se produce cuando una **función lambda** (o función anónima) **captura y utiliza variables del contexto donde fue definida**, incluso **después de que ese contexto haya terminado de ejecutarse**.

Dicho de forma sencilla:

Una lambda puede acceder a **variables externas** a ella, siempre que esas variables sean **efectivamente finales**.

🔑 Idea clave en Java

En **Java**, una lambda puede **leer** variables locales del método que la contiene, con una condición muy importante:

👉 **La variable debe ser *final* o *efectivamente final***

Esto significa:

- No es necesario escribir `final`
- Pero **no puede modificarse después de su inicialización**

Esto garantiza:

- Seguridad en concurrencia
- Coherencia del cierre

✅ Ejemplo simple de closure en Java

Lambda accediendo a una variable local

```
import java.util.function.Function;

public class Main {
    public static void main(String[] args) {

        String sufijo = "!!!"; // variable local (efectivamente final)

        Function<String, String> enfatizar =
```

```
s -> s + sufijo;

    System.out.println(enfatizar.apply("Hola"));
}
}
```

Salida

Hola!!!

👉 Aquí:

- **sufijo** no está definida dentro de la lambda
- La lambda la **captura del entorno**
- Eso es un **closure**

✗ Qué NO se permite (muy típico de examen)

```
String sufijo = "!";
Function<String, String> f = s -> s + sufijo;
sufijo = "??"; // ✗ ERROR de compilación
```

✗ Error porque **sufijo** ya no es efectivamente final.

✓ Modificación del ejemplo anterior: concatenar una cadena externa

Partimos del método **transformar** que ya teníamos:

```
public static String transformar(
    String texto,
    Function<String, String> transformadora) {

    return transformadora.apply(texto);
}
```

✓ Uso de **transformar** con una **lambda** que **captura una variable externa**

```
import java.util.function.Function;

public class Main {
```

```
public static void main(String[] args) {  
  
    String sufijo = " -- FIN"; // variable local capturada (closure)  
  
    String resultado = Util.transformar(  
        "Mensaje",  
        s -> s + sufijo  
    );  
  
    System.out.println(resultado);  
}
```

Salida

```
Mensaje -- FIN
```

Qué está ocurriendo exactamente

- `sufijo` está definido **fuera de la lambda**
- La lambda lo **captura**
- `transformar` **no sabe nada** de `sufijo`
- El entorno de ejecución de la lambda **conserva esa referencia**

👉 Eso es exactamente un **cierre**.

Por qué Java impone la regla de “efectivamente final”

Java obliga a esta restricción para:

- Evitar incoherencias de estado
- Facilitar la ejecución concurrente
- Simplificar el modelo mental (especialmente con hilos)

En lenguajes como JavaScript o Python, esto es más flexible, pero también más propenso a errores sutiles.

Comparación rápida con JavaScript (para entender mejor)

En JavaScript esto sería válido:

```
let sufijo = "!";  
let f = s => s + sufijo;  
sufijo = "??"; // ✅ permitido
```

En Java:

- ❌ No permitido
- ✅ Más seguro

🧠 Resumen perfecto para examen

- Un **closure** ocurre cuando una lambda **accede a variables de su entorno externo**.
- En Java, las lambdas pueden capturar **variables locales efectivamente finales**.
- La variable capturada **no puede modificarse después**.
- Esto permite escribir código funcional **seguro y predecible**.
- Los closures son clave para:
 - callbacks
 - programación funcional
 - APIs como Streams

👉 Frase redonda final:

Un cierre permite que una función lambda recuerde y utilice variables del contexto donde fue creada, incluso fuera de ese contexto, siempre que dichas variables sean efectivamente finales.

8. Reflexiona: ¿en qué se diferencia entonces una función lambda de los punteros a funciones que hay en C?

✅ Punteros a funciones en C

💠 Qué son realmente

Un **puntero a función en C** es **solo una dirección de memoria** donde comienza el código de una función.

- Apunta a **funciones ya definidas**
- No captura contexto
- No tiene estado
- No forma cierres (*closures*)

```
char* (*f)(char*) = convertirAMayusculas;
```

👉 Aquí **f** es simplemente una **dirección**.

💠 Características clave

- ✅ Son **de bajo nivel**
- ✅ Muy próximos al funcionamiento del hardware
- ❌ No capturan variables locales

- ✗ No pueden almacenar estado
- ✗ El programador gestiona todo (tipos, memoria, coherencia)

Ejemplo importante:

En C **no existe el concepto de closure**:

```
int factor = 2;
int (*f)(int) = ... // ✗ imposible capturar "factor"
```

✓ Funciones lambda (Java / JavaScript)

◆ Qué son realmente

Una **función lambda** es un **objeto de alto nivel** que representa:

- Un **comportamiento**
- Posiblemente unido a un **estado capturado** (closure)

En Java, una lambda:

- Implementa una **interfaz funcional**
- Puede capturar variables externas (closure)
- Se gestiona automáticamente (sin punteros explícitos)

◆ Características clave

- ✓ Son **ciudadanos de primera clase**
- ✓ Pueden capturar contexto (closures)
- ✓ Permiten programación declarativa
- ✓ Más seguras (chequeo de tipos, concurrencia)
- ✓ Integradas con librerías (Streams, callbacks, etc.)

Ejemplo de algo **imposible en C** directamente:

```
String sufijo = "!";
Function<String, String> f = s -> s + sufijo;
```

👉 Aquí la lambda **recuerda** el valor de **sufijo**.

✓ Diferencia clave: el **closure**

Característica	Punteros a función (C)	Lambdas (Java)
Capturar variables locales	✗ No	✓ Sí

Característica	Punteros a función (C)	Lambdas (Java)
Closure	✗ No existe	✓ Sí
Estado asociado	✗ No	✓ Opcional
Nivel de abstracción	Bajo	Alto
Seguridad de tipos	Baja	Alta
Memory management	Manual	Automático

👉 Esta es la diferencia fundamental.

✓ Diferencia conceptual profunda

En C:

Una función es código y un puntero apunta a ese código.

En Java (lambdas):

*Una función es un objeto que puede incluir comportamiento **y contexto**.*

Esto hace que:

- En C → Llamadas indirectas
- En Java → **programación funcional real**

✓ ¿Por qué las lambdas no son “punteros a función con otra sintaxis”?

Porque:

- No apuntan necesariamente a una función existente
- Pueden capturar estado
- Se adaptan a interfaces funcionales
- El compilador y la JVM intervienen (optimización, seguridad, concurrencia)

En Java ni siquiera puedes obtener la “dirección” de una lambda.

✓ Ejemplo comparativo muy claro

C (sin closure)

```
int incremento(int x) {  
    return x + 1;  
}
```

No puedes hacer:

```
int n = 1;
int (*f)(int) = x -> x + n; // ❌ imposible
```

Java (con closure)

```
int n = 1;
Function<Integer, Integer> f = x -> x + n;
```

- ✓ Legal
- ✓ Seguro
- ✓ Expresivo

✓ Conclusión clara

Aunque ambas permiten pasar funciones como parámetros, no son equivalentes.

- Un **puntero a función** es una solución **imperativa y de bajo nivel**
- Una **lambda** es una construcción **funcional, de alto nivel**, con cierres y control de tipos

👉 Frase redonda para examen:

Los punteros a función en C permiten invocar código de forma indirecta, mientras que las funciones lambda encapsulan comportamiento y contexto, habilitando closures y programación funcional real.

9. Devolvamos ahora funciones. Creemos ahora una función que sea capaz de crear funciones "descuento". Una función "descuento", decrementa un porcentaje pasado como parámetro. Por simplicidad, usa `Function<Double, Double>` para su tipo. La función `crearDescuento(porcentaje)`, recibe solo el porcentaje de descuento a aplicar y devuelve la función de descuento. Prueba a crear dos descuentos distintos y aplicarlos a una cantidad. Explica la closure en la función descuento.

✓ Objetivo

- Crear una **función generadora de funciones**
- `crearDescuento(porcentaje)`:
 - recibe un porcentaje (por ejemplo `0.10`)
 - **devuelve una función descuento**
- La función descuento:
 - recibe una cantidad (`Double`)
 - devuelve la cantidad con el descuento aplicado
- Usamos `Function<Double, Double>`
- Probamos **dos descuentos distintos**

- Explicamos el **closure** implicado

✓ Función que crea funciones de descuento

```
import java.util.function.Function;

public class Descuentos {

    public static Function<Double, Double> crearDescuento(double porcentaje) {

        return cantidad -> cantidad * (1 - porcentaje);
    }
}
```

✓ Uso: crear y aplicar funciones descuento

```
public class Main {

    public static void main(String[] args) {

        // Creamos dos funciones descuento distintas
        Function<Double, Double> descuento10 =
            Descuentos.crearDescuento(0.10); // 10%

        Function<Double, Double> descuento25 =
            Descuentos.crearDescuento(0.25); // 25%

        double precio = 200.0;

        System.out.println(descuento10.apply(precio));
        System.out.println(descuento25.apply(precio));
    }
}
```

✓ Salida del programa

```
180.0
150.0
```

✓ Qué está ocurriendo exactamente

- 1 `crearDescuento` es una **función de orden superior**

Porque:

-  **recibe datos**
-  **devuelve una función**

```
Function<Double, Double> crearDescuento(...)
```

Esto es una característica central del **paradigma funcional**.

La función descuento es una **lambda**

```
cantidad -> cantidad * (1 - porcentaje)
```

- Recibe `cantidad`
 - Devuelve la cantidad con descuento
 - Implementa `Function<Double, Double>`
-

¿Dónde está el **closure**?

El closure aparece aquí:

```
return cantidad -> cantidad * (1 - porcentaje);
```

♦ `porcentaje`:

- Es una **variable local** de `crearDescuento`
- **No pertenece** a la lambda
- Sin embargo, la lambda **la captura y la usa**

👉 Cada vez que llamas a:

```
crearDescuento(0.10)  
crearDescuento(0.25)
```

se crea:

- una **nueva función**
 - con su propio **valor capturado de porcentaje**
-

Clave del closure

Aunque `crearDescuento` ya ha terminado de ejecutarse, la función devuelta:

- sigue teniendo acceso a `porcentaje`
- conserva su valor
- lo "recuerda"

Eso es exactamente un **closure**.

✓ Visualización conceptual

```
crearDescuento(0.10)
  └─ devuelve función que recuerda porcentaje = 0.10

crearDescuento(0.25)
  └─ devuelve función que recuerda porcentaje = 0.25
```

Cada función descuento es:

- mismo código
- **estado distinto capturado**

✓ Por qué funciona en Java

Porque:

- Las lambdas son **ciudadanos de primera clase**
- Java permite **closures** sobre variables:
 - `final`
 - o **efectivamente finales**

✦ En este ejemplo:

```
double porcentaje
```

nunca se modifica → es **efectivamente final** ✓

✓ Comparación mental rápida (muy de examen)

Concepto	Ejemplo
Función	<code>cantidad -> cantidad * (1 - porcentaje)</code>
Función de orden superior	<code>crearDescuento</code>
Closure	Captura de <code>porcentaje</code>

Concepto	Ejemplo
Estado capturado	0.10, 0.25
Seguridad	Chequeo en compilación

✓ Resumen perfecto para examen

- `crearDescuento` es una **función de orden superior** que devuelve funciones.
- La función descuento es una **lambda** de tipo `Function<Double, Double>`.
- Cada función creada **captura el valor de porcentaje**.
- Ese valor se conserva aunque el método haya terminado.
- Esto se llama **closure**.
- Gracias a esto, se pueden crear funciones **parametrizadas por estado**, de forma segura y expresiva.

👉 Frase redonda final:

Un closure permite que una función devuelta recuerde y utilice las variables del contexto en el que fue creada, incluso después de que dicho contexto haya finalizado.

10. En Java, que es un lenguaje con comprobación estática de tipos, donde los tipos se declaran, toda función lambda tiene un tipo, que se conoce como **interfaz funcional**. ¿Qué es una **interfaz funcional**? ¿Qué requisitos tiene?

En Java, una **función lambda** no existe "aislada": **siempre tiene un tipo**, y ese tipo es una **interfaz funcional**.

✓ ¿Qué es una **interfaz funcional**?

Una **interfaz funcional** es una **interfaz que declara exactamente un único método abstracto**.

Ese método abstracto define la **firma** (parámetros y tipo de retorno) que deben cumplir las funciones lambda que se asignen a ese tipo.

👉 En otras palabras:

Una interfaz funcional define el tipo de una función lambda en Java.

Ejemplo simple:

```
@FunctionalInterface
public interface Transformador {
    String transformar(String s);
}
```

Una lambda compatible sería:

```
Transformador t = s -> s.toUpperCase();
```

La lambda implementa el **único método abstracto** de la interfaz.

✓ Requisitos de una interfaz funcional

Para que una interfaz sea funcional en Java, debe cumplir **todas** estas condiciones:

◆ 1. Debe tener **exactamente un método abstracto**

✓ Correcto:

```
interface Operacion {  
    int aplicar(int a, int b);  
}
```

✗ Incorrecto (dos métodos abstractos):

```
interface Mal {  
    int a(int x);  
    int b(int x); // ✗  
}
```

◆ 2. Puede tener **métodos default**

Los métodos **default** **NO cuentan** como métodos abstractos.

```
interface Operacion {  
    int aplicar(int a, int b);  
  
    default void info() {  
        System.out.println("Operación binaria");  
    }  
}
```

✓ Sigue siendo funcional.

◆ 3. Puede tener **métodos static**

Los métodos **static** tampoco afectan a la condición de interfaz funcional.

```
interface Operacion {  
    int aplicar(int a, int b);  
  
    static void ayuda() {  
        System.out.println("Uso de la operación");  
    }  
}
```

◆ 4. Puede heredar métodos abstractos de `Object`

Métodos como:

- `toString`
- `equals`
- `hashCode`

no cuentan, aunque aparezcan en la interfaz.

```
interface Ejemplo {  
    boolean equals(Object o); // heredado de Object  
    String procesar(String s);  
}
```

✅ Sigue siendo funcional.

✅ La anotación `@FunctionalInterface`

Java ofrece la anotación:

```
@FunctionalInterface
```

◆ ¿Es obligatoria?

❌ No es obligatoria

✅ Pero **muy recomendable**

◆ ¿Para qué sirve?

- ✅ Hace explícita la intención de la interfaz
- ✅ El compilador verifica que **solo haya un método abstracto**
- ✅ Evita errores accidentales al modificar la interfaz

Ejemplo:

```
@FunctionalInterface
public interface Transformador {
    String transformar(String s);
}
```

Si alguien añade otro método abstracto, el compilador **fallará**.

✓ Ejemplos de interfaces funcionales en la API estándar

Java incluye muchas **interfaces funcionales predefinidas**, especialmente en `java.util.function`:

Interfaz	Firma del método
<code>Function<T,R></code>	<code>R apply(T t)</code>
<code>Predicate<T></code>	<code>boolean test(T t)</code>
<code>Consumer<T></code>	<code>void accept(T t)</code>
<code>Supplier<T></code>	<code>T get()</code>
<code>UnaryOperator<T></code>	<code>T apply(T t)</code>

Ejemplo con lambda:

```
Function<String, Integer> longitud = s -> s.length();
```

✓ Relación con las funciones lambda

Una lambda **no tiene tipo por sí misma**.

El tipo lo proporciona el **contexto**, es decir, la **interfaz funcional** a la que se asigna:

```
Function<String, String> f = s -> s.toUpperCase();
```

Aquí:

- `Function<String, String>` es el tipo
- La lambda implementa `apply(String)`

👉 Esto se llama **tipado por contexto** (*target typing*).

✓ Comparación conceptual (muy de examen)

Concepto	Papel
Lambda	Implementación sin nombre
Interfaz funcional	Tipo de la lambda
Método abstracto	Firma de la función
<code>@FunctionalInterface</code>	Garantía del contrato

Resumen perfecto para examen

- Una **interfaz funcional** es una interfaz con **un único método abstracto**.
- Define el **tipo** de una función lambda en Java.
- Puede contener métodos `default` y `static`.
- Los métodos de `Object` no cuentan.
- La anotación `@FunctionalInterface` no es obligatoria, pero garantiza la corrección.
- Las lambdas siempre se asignan a **interfaces funcionales**.
- Este mecanismo permite programación funcional con **comprobación estática de tipos**.

Frase redonda final:

En Java, una función lambda siempre tiene como tipo una interfaz funcional, que define de forma explícita la firma del comportamiento que la lambda implementa.

11. Creemos una interfaz funcional a mano. Por ejemplo, define la interfaz funcional del ejemplo que transforma la cadena en otra. Llámale `Transformador`, que define una función que convierte una cadena de texto (`String`) en otra (`String`).

Interfaz funcional `Transformador`

Queremos una interfaz que represente **una función que recibe un `String` y devuelve otro `String`**. Eso encaja perfectamente con una **interfaz funcional**.

Definición en Java

```
@FunctionalInterface
public interface Transformador {

    String transformar(String texto);
}
```

Por qué esta interfaz es funcional

Cumple **todos los requisitos**:

- ☒ Tiene **un único método abstracto**
- ☒ El método define la firma `String → String`
- ☒ Puede usarse como tipo de una lambda
- ☒ La anotación `@FunctionalInterface` garantiza el contrato

☒ Uso con una función lambda

```
public class Main {  
    public static void main(String[] args) {  
  
        Transformador aMayusculas =  
            s -> s.toUpperCase();  
  
        System.out.println(aMayusculas.transformar("Hola mundo"));  
    }  
}
```

Salida:

```
HOLA MUNDO
```

La lambda implementa automáticamente el método:

```
String transformar(String texto);
```

☒ Uso con el método `transformar` visto antes

```
public class Util {  
  
    public static String transformar(  
        String texto,  
        Transformador t) {  
  
        return t.transformar(texto);  
    }  
}
```

Y la llamada:

```
String resultado = Util.transformar(  
    "Hola mundo",
```

```
s -> new StringBuilder(s).reverse().toString()  
);  
  
System.out.println(resultado);
```

Salida:

```
odnum aloH
```

Relación con `Function<String, String>`

Tu interfaz `Transformador` es **equivalente conceptualmente** a:

```
Function<String, String>
```

La diferencia es:

- `Transformador` → interfaz **semántica y específica del dominio**
- `Function<String, String>` → interfaz **genérica**

En diseño real:

- APIs públicas → suelen usar interfaces propias (`Transformador`)
- Utilidades generales → usan `Function`

Resumen perfecto para examen

- Una **interfaz funcional** define el tipo de una lambda.
- `Transformador` representa una función `String → String`.
- Debe tener **exactamente un método abstracto**.
- La anotación `@FunctionalInterface` garantiza que lo siga siendo.
- Las lambdas implementan directamente ese método.
- Permite pasar comportamiento como parámetro con chequeo estático.

Frase redonda final:

Una interfaz funcional define el tipo de una función lambda en Java, especificando de forma explícita la firma del comportamiento esperado.

12. Ahora hagamos la interfaz funcional algo más genérica y empleando generics, para que permita definir un `Transformador` de un tipo en otro. Pon un ejemplo de un transformador que redondea un `Double` en un `Integer`.

✓ Interfaz funcional genérica Transformador<T, R>

Queremos que el transformador **no esté limitado a** `String → String`, sino que permita expresar:

“Transformar un valor de tipo **T** en otro de tipo **R**”.

Definición de la interfaz funcional

```
@FunctionalInterface
public interface Transformador<T, R> {

    R transformar(T valor);
}
```

✓ Por qué esta interfaz es correcta

- ✓ Es **funcional**: solo tiene un método abstracto
- ✓ Usa **genéricos** (`T, R`)
- ✓ Permite transformar **cualquier tipo en cualquier otro**
- ✓ Es equivalente conceptualmente a `Function<T, R>`
- ✓ Es más **expresiva semánticamente** que `Function`

✓ Ejemplo: transformar `Double → Integer` (redondeo)

Ahora definimos un transformador que:

- recibe un `Double`
- devuelve un `Integer`
- redondea el valor

Lambda usando la interfaz `Transformador`

```
public class Main {

    public static void main(String[] args) {

        Transformador<Double, Integer> redondear =
            d -> (int) Math.round(d);

        Integer resultado = redondear.transformar(3.7);

        System.out.println(resultado);
    }
}
```

Salida

4

✓ Qué está pasando exactamente

- `Transformador<Double, Integer>` fija:
 - `T = Double`
 - `R = Integer`
- La lambda:

```
d -> (int) Math.round(d)
```

implementa:

```
Integer transformar(Double valor);
```

✓ El compilador:

- verifica tipos en **compilación**
- evita castings inseguros
- garantiza coherencia del diseño

✓ Comparación con `Function<T, R>`

Tu interfaz:

```
Transformador<Double, Integer>
```

es funcionalmente equivalente a:

```
Function<Double, Integer>
```

La diferencia es **conceptual y de diseño**:

Aspecto	Transformador	Function
Semántica	Clara y de dominio	Genérica
Legibilidad	✓ Alta	⚠ Neutral
Reutilización	Similar	Similar

Aspecto	Transformador	Function
Uso en APIs	Muy común	Muy común

👉 En código real:

- **APIs de dominio** → interfaces propias (**Transformador**)
- **Utilidades generales** → **Function**

✅ Ejemplo adicional de reutilización

La misma interfaz sirve para muchos casos:

```
Transformador<Integer, String> aTexto =  
    i -> "Número: " + i;  
  
Transformador<String, Integer> longitud =  
    s -> s.length();
```

💬 Resumen perfecto para examen

- Una **interfaz funcional genérica** puede parametrizar tipos de entrada y salida.
- **Transformador<T, R>** representa una función de $T \rightarrow R$.
- Permite definir transformaciones fuertemente tipadas.
- El compilador garantiza seguridad de tipos en compilación.
- Un ejemplo de uso es un transformador **Double** → **Integer** que redondea.
- Es equivalente a **Function<T, R>**, pero más expresivo semánticamente.

👉 **Frase redonda final:**

Las interfaces funcionales genéricas permiten definir transformaciones entre tipos arbitrarios con seguridad de tipos y gran expresividad.

13. **Transformador**, en su versión genérica, parece muy útil y reutilizable, hasta el punto de que es igual a una interfaz funcional que ya hay, que es **Function<T, R>**. Muestra las interfaces funcionales predefinidas que hay en Java.

✅ Interfaces funcionales predefinidas en Java (**java.util.function**)

◆ 1. **Function<T, R>**

Transforma un valor en otro

```
R apply(T t)
```

✦ Uso típico: transformar datos

```
Function<String, Integer> longitud = s -> s.length();
```

→ Es exactamente el equivalente genérico de tu `Transformador<T, R>`.

◆ 2. `Predicate<T>`

Evalúa una condición

```
boolean test(T t)
```

✦ Uso típico: filtros

```
Predicate<Integer> esPositivo = n -> n > 0;
```

◆ 3. `Consumer<T>`

Consume un valor (no devuelve nada)

```
void accept(T t)
```

✦ Uso típico: acciones, efectos laterales

```
Consumer<String> imprimir = s -> System.out.println(s);
```

◆ 4. `Supplier<T>`

Produce valores (no recibe nada)

```
T get()
```

✦ Uso típico: generación perezosa

```
Supplier<Double> aleatorio = () -> Math.random();
```

◆ 5. UnaryOperator<T>

Transforma un valor en otro del mismo tipo

```
T apply(T t)
```

✦ Equivale a `Function<T, T>`

```
UnaryOperator<Integer> cuadrado = x -> x * x;
```

◆ 6. BinaryOperator<T>

Opera sobre dos valores del mismo tipo y devuelve ese tipo

```
T apply(T a, T b)
```

✦ Equivale a `Function<T, Function<T, T>>`

```
BinaryOperator<Integer> suma = (a, b) -> a + b;
```

◆ 7. Interfaces funcionales para tipos primitivos

Para **evitar boxing/unboxing**, Java define versiones especializadas.

👉 Para `int`

Interfaz	Método
<code>IntFunction<R></code>	<code>R apply(int value)</code>
<code>IntPredicate</code>	<code>boolean test(int value)</code>
<code>IntConsumer</code>	<code>void accept(int value)</code>
<code>IntSupplier</code>	<code>int getAsInt()</code>
<code>IntUnaryOperator</code>	<code>int applyAsInt(int operand)</code>

Interfaz	Método
<code>IntBinaryOperator</code>	<code>int applyAsInt(int a, int b)</code>

Ejemplo:

```
IntPredicate esPar = n -> n % 2 == 0;
```

→ Existen equivalentes para:

- `double` → `DoubleFunction`, `DoublePredicate`, etc.
- `long` → `LongFunction`, `LongPredicate`, etc.

◆ 8. `BiFunction<T, U, R>`

Transforma dos valores en uno

```
R apply(T t, U u)
```

✦ Uso típico: combinaciones

```
BiFunction<Integer, Integer, Integer> suma = (a, b) -> a + b;
```

◆ 9. `BiPredicate<T, U>`

Condición sobre dos valores

```
boolean test(T t, U u)
```

```
BiPredicate<String, String> iguales = (a, b) -> a.equals(b);
```

◆ 10. `BiConsumer<T, U>`

Consume dos valores

```
void accept(T t, U u)
```

```
BiConsumer<String, Integer> imprimir =
    (s, n) -> System.out.println(s + ": " + n);
```

Resumen estructurado (muy de examen)

♦ Clasificación funcional

Propósito	Interfaz
Transformar	<code>Function<T,R></code>
Condición	<code>Predicate<T></code>
Consumir	<code>Consumer<T></code>
Producir	<code>Supplier<T></code>
Transformar mismo tipo	<code>UnaryOperator<T></code>
Operar dos valores	<code>BinaryOperator<T></code>
Transformar dos valores	<code>BiFunction<T,U,R></code>

✓ Conclusión clave

- Java proporciona **interfaces funcionales estándar** para los patrones funcionales más comunes.
- `Function<T, R>` **equivale exactamente** a tu `Transformador<T, R>`.
- Usar interfaces estándar:
 - ✓ mejora interoperabilidad
 - ✓ evita duplicación
 - ✓ integra mejor con Streams y APIs modernas
- Interfaces propias (`Transformador`) son útiles cuando:
 - el nombre aporta **semántica de dominio**
 - se quiere un diseño más expresivo

✓ Frase redonda para examen

Java incluye un conjunto amplio de interfaces funcionales predefinidas en `java.util.function`, siendo `Function<T,R>` el arquetipo general para representar transformaciones entre tipos.

14. Vamos a ver ejemplos expresivos de funcional en Java. Estudiemos el `List.forEach`, como versión funcional del bucle `for`. Emplea el `forEach` para recorrer una lista de `Integer` y que muestre un mensaje si el entero es positivo.

✓ Idea clave: `forEach` en Java

Desde **Java 8**, las colecciones incluyen el método:

```
void forEach(Consumer<? super T> action)
```

- Recorre la colección
- Aplica una **función lambda** a cada elemento
- Es una forma **funcional y declarativa** de iterar

✅ Ejemplo: recorrer una `List<Integer>` y mostrar mensaje si es positivo

Código completo

```
import java.util.Arrays;
import java.util.List;

public class Main {
    public static void main(String[] args) {

        List<Integer> numeros = Arrays.asList(-3, 5, 0, 7, -1, 10);

        numeros.forEach(n -> {
            if (n > 0) {
                System.out.println(n + " es positivo");
            }
        });
    }
}
```

✅ Salida

```
5 es positivo
7 es positivo
10 es positivo
```

🔍 Qué está pasando

- `numeros.forEach(...)` recorre la lista
- La lambda `n -> { ... }` es un **Consumer**
- Para cada elemento:
 - se evalúa la condición
 - se ejecuta la acción si procede

👉 No hay índices, no hay variables auxiliares, solo **qué hacer con cada elemento**.

✅ Comparación con el bucle **for** tradicional

Imperativo (antes de Java 8)

```
for (Integer n : numeros) {  
    if (n > 0) {  
        System.out.println(n + " es positivo");  
    }  
}
```

Funcional (Java 8+)

```
numeros.forEach(n -> {  
    if (n > 0) {  
        System.out.println(n + " es positivo");  
    }  
});
```

- ✅ Mismo comportamiento
 - ✅ Estilo más declarativo
 - ✅ Uso de funciones como ciudadanos de primera clase
-

✅ Versión aún más funcional (adelanto)

Aunque aquí el ejercicio pide **forEach**, en estilo funcional puro suele hacerse:

```
numeros.stream()  
    .filter(n -> n > 0)  
    .forEach(n -> System.out.println(n + " es positivo"));
```

(Esto pertenece ya a **Streams**, siguiente paso natural del tema).

🗨️ Resumen para examen

- **forEach** es la versión funcional del bucle **for**
- Recibe un **Consumer<T>**
- Permite recorrer colecciones usando **lambdas**
- Hace el código más expresivo y declarativo
- Es una puerta de entrada a programación funcional en Java

👉 **Frase redonda final:**

forEach permite recorrer colecciones aplicando una función a cada elemento, sustituyendo el bucle imperativo por un enfoque funcional.

15. Repasando el tema de genericidad, fíjate en la firma de `forEach`, ¿por qué se usa `Consumer<? super T>` y no `Consumer<T>`? Explica qué significa **PECS**, y explícalo para el caso de mejorar el ejemplo del método `transformar` la hora de definir el tipo de la función transformadora.

✓ Firma de `forEach`

En Java, `forEach` se declara así (simplificado):

```
void forEach(Consumer<? super T> action)
```

La pregunta es:

¿Por qué se usa `Consumer<? super T>` y no `Consumer<T>`?

La respuesta está en **PECS**.

✓ ¿Qué es **PECS**?

PECS es una regla mnemotécnica que resume cómo usar **wildcards** con genéricos:

Producer → **extends**

Consumer → **super**

Es decir:

- Si una estructura **produce valores** → usa `? extends T`
- Si una estructura **consume valores** → usa `? super T`

✓ Aplicación directa a `forEach`

🔍 ¿Qué hace `forEach`?

`forEach`:

- **NO produce** elementos
- **Consume** elementos de la colección
- Pasa cada elemento de tipo `T` **como argumento** a la función

Ejemplo:

```
List<Integer> lista = ...  
lista.forEach(n -> System.out.println(n));
```

→ El **Consumer** **recibe** (**consume**) valores de tipo **T**.

✓ Por qué **Consumer<? super T>**

Si usamos:

```
Consumer<? super T>
```

permitimos pasar:

- **Consumer<T>**
- **o cualquier consumidor de un supertipo de T**

Ejemplo:

```
Consumer<Object> imprimir = o -> System.out.println(o);  
  
List<Integer> numeros = List.of(1, 2, 3);  
numeros.forEach(imprimir); // ✓ permitido
```

Esto es totalmente seguro, porque:

- Un **Consumer<Object>** **puede aceptar Integers**
 - **Object** es supertipo de **Integer**
-

✗ Qué pasaría con **Consumer<T>**

Si la firma fuera:

```
void forEach(Consumer<T> action)
```

entonces esto **NO compilaría**:

```
Consumer<Object> imprimir = o -> System.out.println(o);  
numeros.forEach(imprimir); // ✗ ERROR
```

👉 Se perdería **flexibilidad sin ganar seguridad**.

✦ Conclusión clave:

`Consumer<? super T>` permite **contravarianza controlada**, aumentando reutilización y manteniendo seguridad.

✓ Resumen visual de `forEach`

Elemento	Rol	PECS
Lista (<code>List<T></code>)	Produce T	<code>extends</code> (en Streams)
Consumer	Consume T	<code>super</code> ✓

✓ Aplicación de **PECS** al método `transformar`

Recordemos el método original:

```
public static String transformar(
    String texto,
    Function<String, String> transformadora) {

    return transformadora.apply(texto);
}
```

🔍 ¿Qué papel juega la función transformadora?

La función:

- **consume** un `String` (entrada)
- **produce** otro `String` (salida)

Pero desde el punto de vista del método `transformar`:

- Solo **le pasamos** un `String`
- Nos interesa que la función **pueda aceptar `String` o algo más genérico**

✓ Mejora usando PECS

Aplicamos la regla:

- **consume** `String` → `super String`
- **produce** `String` → `extends String`

Queda así:

```
public static String transformar(
    String texto,
    Function<? super String, ? extends String> transformadora) {

    return transformadora.apply(texto);
}
```

✓ Qué ganamos con esta firma

✓ Más flexibilidad

Ahora podemos pasar, por ejemplo:

```
Function<Object, String> f = o -> o.toString().toUpperCase();
```

Y usarla sin problemas:

```
String r = transformar("hola", f); // ✓ correcto
```

Porque:

- `Object` es **super** de `String` → consume seguro
- Devuelve algo que es **subtipo de** `String`

✓ Relación directa con PECS

Parte de la función	Rol	Wildcard
Parámetro de entrada	Consumer	? <code>super String</code>
Tipo de retorno	Producer	? <code>extends String</code>

👉 **PECS aplicado a funciones.**

✓ Comparación final

✗ Sin PECS (más restrictivo)

```
Function<String, String>
```

- Correcto

- Menos reutilizable

✓ Con PECS (más flexible y seguro)

```
Function<? super String, ? extends String>
```

- Más general
- 100 % seguro en compilación
- Mejor diseño de API

🧠 Resumen perfecto para examen

- `forEach` usa `Consumer<? super T>` porque el `Consumer` **consume** elementos.
- Esto sigue la regla **PECS: Consumer Super**.
- Usar `? super T` permite pasar consumidores de tipos más generales.
- PECS se aplica también a funciones:
 - entrada → `super`
 - salida → `extends`
- Aplicando PECS a `transformar`, se obtiene una API más flexible y correcta.

👉 Frase redonda final:

PECS permite expresar correctamente la variancia en genéricos: los productores usan `extends` y los consumidores usan `super`, como ocurre en `forEach` y en funciones transformadoras bien diseñadas.

16. Referencias a métodos. Podemos obtener una referencia a métodos de objetos o clases. Pon un ejemplo en JavaScript y en Java, de una clase `Persona` con un método `saludar`. En el código principal, crea una `Persona` con un nombre, y obtén una referencia a su método `saludar` en una variable local. Invoca `saludar` con esa referencia a su método `saludar`.

✓ ¿Qué es una **referencia a método**?

Una **referencia a método** consiste en **tratar un método como un valor**, asignándolo a una variable para poder **invocarlo más tarde**, sin llamarlo inmediatamente.

Esto refuerza la idea de:

- funciones/métodos como **ciudadanos de primera clase**
- programación **funcional y expresiva**

✓ Ejemplo en **JavaScript**

En JavaScript, los métodos son funciones, pero **hay que tener cuidado con `this`**.

Clase `Persona`

```
class Persona {  
  constructor(nombre) {  
    this.nombre = nombre;  
  }  
  
  saludar() {  
    console.log("Hola, soy " + this.nombre);  
  }  
}
```

Obtener una referencia al método `saludar`

⚠ En JavaScript, si pasamos el método directamente, **se pierde el contexto (`this`)**.
Por eso usamos `.bind(this)`.

```
const p = new Persona("Ana");  
  
// Referencia al método, ligada al objeto  
const saludo = p.saludar.bind(p);  
  
// Invocación a través de la referencia  
saludo();
```

Salida

```
Hola, soy Ana
```

Explicación clave (JavaScript)

- `p.saludar` → es una función
- `bind(p)` → fija el valor de `this`
- `saludo` → referencia al método ya ligado
- `saludo()` → ejecuta `saludar` correctamente

👉 Sin `bind`, `this.nombre` sería `undefined`.



Ejemplo en Java

En Java, las referencias a métodos forman parte del lenguaje desde **Java 8** y **son seguras por diseño**.

Clase **Persona**

```
public class Persona {  
  
    private final String nombre;  
  
    public Persona(String nombre) {  
        this.nombre = nombre;  
    }  
  
    public void saludar() {  
        System.out.println("Hola, soy " + nombre);  
    }  
}
```

Obtener una referencia al método **saludar**

Como **saludar**:

- no recibe parámetros
- no devuelve nada

usamos la interfaz funcional **Runnable**.

```
public class Main {  
  
    public static void main(String[] args) {  
  
        Persona p = new Persona("Ana");  
  
        // Referencia al método de instancia  
        Runnable saludo = p::saludar;  
  
        // Invocación a través de la referencia  
        saludo.run();  
    }  
}
```

Salida

Hola, soy Ana

🔍 Explicación clave (Java)

- `p::saludar` es una **referencia a un método de instancia**
- Se asigna a `Runnable` porque:

```
void run();
```

coincide con:

```
void saludar();
```

- `saludo.run()` invoca indirectamente `p.saludar()`

👉 **No hay problemas de contexto** como en JavaScript.

✅ Comparación JavaScript vs Java

Aspecto	JavaScript	Java
Método como valor	✅ Sí	✅ Sí
Sintaxis	<code>obj.metodo.bind(obj)</code>	<code>obj::metodo</code>
Control de contexto	Manual (<code>bind</code>)	Automático
Tipado estático	❌ No	✅ Sí
Interfaz funcional	❌ No necesaria	✅ Obligatoria

✅ Tipos de referencias a métodos en Java (extra útil)

En Java existen cuatro formas:

```
obj::metodo           // método de instancia
Clase::metodoEstatico // método estático
Clase::new            // constructor
Clase::metodo         // método de instancia vía parámetro
```

En este ejercicio usamos:

```
p::saludar
```

Resumen perfecto para examen

- Una **referencia a método** permite invocar un método indirectamente.
- En **JavaScript**, hay que ligar el contexto (**bind**) para no perder **this**.
- En **Java**, las referencias a métodos (**::**) son seguras y tipadas.
- En Java, una referencia a método siempre se asigna a una **interfaz funcional**.
- Es una forma más expresiva y clara de trabajar con comportamiento.

Frase redonda final:

Las referencias a métodos permiten tratar métodos como valores, reforzando el estilo funcional y facilitando la reutilización de comportamiento.

17. ¿Qué tipos de referencias a método se pueden hacer en Java? Pon un ejemplo de referencia a método estático, a constructor, a método de instancia de una instancia concreta y a método de instancia sobre cualquier instancia.

En **Java**, las **referencias a métodos** (**::**) permiten usar métodos (o constructores) como valores, siempre que **encajen con una interfaz funcional**. El lenguaje define **cuatro tipos principales** de referencias.

Tipos de referencias a método en Java

1 Referencia a método estático

Forma: `Clase::metodoEstatico`

Se usa cuando el método es **static**.

```
import java.util.function.Function;

public class Util {
    public static String aMayusculas(String s) {
        return s.toUpperCase();
    }
}

public class Main {
    public static void main(String[] args) {
        Function<String, String> f = Util::aMayusculas;
        System.out.println(f.apply("hola"));
    }
}
```

✓ La referencia equivale a la lambda: `s -> Util.aMayusculas(s)`.

2 Referencia a constructor

Forma: `Clase::new`

Se usa para crear objetos.

```
import java.util.function.Supplier;

public class Persona {
    private final String nombre;
    public Persona() {
        this.nombre = "Anónima";
    }
    public Persona(String nombre) {
        this.nombre = nombre;
    }
    public String getNombre() { return nombre; }
}

public class Main {
    public static void main(String[] args) {
        Supplier<Persona> creador = Persona::new; // constructor sin parámetros
        Persona p = creador.get();
        System.out.println(p.getNombre());
    }
}
```

Con parámetros:

```
import java.util.function.Function;

Function<String, Persona> creador = Persona::new; // constructor con String
Persona p = creador.apply("Ana");
```

3 Referencia a método de instancia de un objeto concreto

Forma: `objeto::metodo`

El método pertenece a una **instancia ya creada**.

```
public class Persona {
    private final String nombre;
    public Persona(String nombre) { this.nombre = nombre; }
    public void saludar() {
```

```

        System.out.println("Hola, soy " + nombre);
    }
}

public class Main {
    public static void main(String[] args) {
        Persona p = new Persona("Ana");
        Runnable saludo = p::saludar;
        saludo.run();
    }
}

```

✓ Equivale a la lambda: `() -> p.saludar()`.

4 Referencia a método de instancia sobre cualquier instancia

Forma: `Clase::metodo`

Aquí el objeto **se proporciona como parámetro implícito**.

```

import java.util.function.Function;

public class Main {
    public static void main(String[] args) {
        Function<String, String> f = String::toUpperCase;
        System.out.println(f.apply("hola"));
    }
}

```

✓ Equivale a la lambda: `s -> s.toUpperCase()`.

Otro ejemplo con dos parámetros (el receptor es el primero):

```

import java.util.function.BiPredicate;

BiPredicate<String, String> iguales = String::equals;
System.out.println(iguales.test("a", "a"));

```

Equivale a: `(a, b) -> a.equals(b)`.



Resumen rápido (ideal para examen)

- **Método estático:** `Clase::metodoEstatico`
- **Constructor:** `Clase::new`
- **Método de instancia (objeto concreto):** `objeto::metodo`
- **Método de instancia (cualquier instancia):** `Clase::metodo`

👉 Todas las referencias a métodos **deben encajar con una interfaz funcional** (misma firma).

Frase redonda final:

Las referencias a métodos son una forma concisa y tipada de tratar métodos y constructores como valores, integrando la programación funcional en Java.

18. Otro ejemplo expresivo. Ordena una lista de **Persona**, cada persona tiene un nombre y una edad (de tipo entero). Ordena la lista de **Persona** con **Collections.sort**, pasándole como comparador una expresión lambda que compare la edad de ambas personas y si tienen la misma edad, se ordene por orden alfabético del nombre. Crea dos versiones: Una con la función de comparación hecha manualmente, y otra empleando **Comparator**.

✓ Clase **Persona**

```
public class Persona {  
    private String nombre;  
    private int edad;  
  
    public Persona(String nombre, int edad) {  
        this.nombre = nombre;  
        this.edad = edad;  
    }  
  
    public String getNombre() {  
        return nombre;  
    }  
  
    public int getEdad() {  
        return edad;  
    }  
  
    @Override  
    public String toString() {  
        return nombre + " (" + edad + ")";  
    }  
}
```

✓ Datos de prueba

```
import java.util.ArrayList;  
import java.util.List;  
  
List<Persona> personas = new ArrayList<>();  
personas.add(new Persona("Ana", 30));  
personas.add(new Persona("Luis", 25));
```

```
personas.add(new Persona("Carlos", 30));
personas.add(new Persona("Bea", 25));
```

● Versión 1: Comparador **manual** con lambda

Aquí implementamos **toda la lógica de comparación a mano** dentro de la lambda.

```
import java.util.Collections;

Collections.sort(personas, (p1, p2) -> {
    if (p1.getEdad() != p2.getEdad()) {
        return p1.getEdad() - p2.getEdad(); // primero por edad
    } else {
        return p1.getNombre().compareTo(p2.getNombre()); // luego por nombre
    }
});
```

✚ Explicación

- Se compara primero la **edad**
- Si la edad es igual, se compara el **nombre alfabéticamente**
- La lambda implementa la interfaz funcional `Comparator<Persona>`

● Versión 2: Usando `Comparator` (forma recomendada y más funcional)

Aquí usamos los **métodos auxiliares** de `Comparator`, que dan un código más limpio y declarativo.

```
import java.util.Comparator;

Collections.sort(
    personas,
    Comparator.comparingInt(Persona::getEdad)
                .thenComparing(Persona::getNombre)
);
```

✚ Explicación

- `comparingInt(Persona::getEdad)` → criterio principal
- `thenComparing(Persona::getNombre)` → criterio secundario
- Usa **referencias a métodos**
- Es más expresivo y menos propenso a errores

✓ Mostrar el resultado

```
personas.forEach(System.out::println);
```

Salida esperada:

```
Bea (25)
Luis (25)
Ana (30)
Carlos (30)
```

🗨 Comparación clara entre ambas versiones

Aspecto	Comparador manual	Comparator
Claridad	Media	✓ Alta
Verbosidad	Alta	✓ Baja
Funcional	Parcial	✓ Total
Referencias a método	✗ No	✓ Sí
Recomendado	⚠ Aceptable	✓ Sí

✓ Resumen perfecto para examen

- `Collections.sort` recibe un `Comparator`, que es una **interfaz funcional**
- Puede implementarse con una **lambda**
- La comparación puede hacerse:
 - manualmente con `if`
 - o usando `Comparator.comparing` y `thenComparing`
- La segunda opción es **más declarativa y funcional**
- Se integra perfectamente con **lambdas y referencias a métodos**

👉 Frase redonda para cerrar:

El uso de `Comparator` junto con expresiones lambda permite definir criterios de ordenación complejos de forma clara, segura y funcional.